Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Уфимский университет науки и технологий»

Кафедра Высокопроизводительных вычислений и дифференциальных уравнений

	1	2	3	4	5	6	7	8	9	10
100										
90										
80										
70										
60										
50										
40										
30										
20										
10										
0										

ОТЧЕТ

По лабораторной работе №1

Параллельное вычисление суммы числового ряда

По дисциплине **...**

Группа	Фамилия И.О.	Подпись	Дата	Оценка
ПМ-401Б				
Студент	Халитов А.Ю.			—
Принял				

Уфа 2026

1 Цель работы

Для многопроцессорных вычислительных систем с распределенной памятью на примере задачи параллельного вычисления суммы числового ряда научиться программно реализовывать простейшие параллельные вычислительные алгоритмы с использованием базовых функций библиотеки MPI, а также проводить анализ их эффективности.

2 Постановка задачи

Требуется вычислить сумму числового ряда:

$$S = \sum_{n=1}^{N} a_n = a_1 + a_2 + \cdots + a_N, \quad a_n \in \mathbb{R},\ n \in \mathbb{N}$$

В распоряжении находится многопроцессорная вычислительная система (МВС) с P процессорами (ядрами).

3 Алгоритм параллельного вычисления

Для реализации параллельного алгоритма используются базовые функции MPI: `MPI_Init`, `MPI_Finalize`, `MPI_Comm_size`, `MPI_Comm_rank`, `MPI_Send`, `MPI_Recv`, `MPI_Wtime`, `MPI_Barrier`.

Схема работы параллельной программы:

1. Ввод-вывод данных осуществляется через нулевой процесс (`Rank` $= 0$), который передает остальным процессам число членов ряда N, полученное из аргументов командной строки.

2. Время начала выполнения параллельного участка программы замеряется с помощью функции `MPI_Wtime()`.

3. Каждый процесс определяет количество элементов для суммирования:

   $$h = \lfloor N/P \rfloor, \qquad m = N \bmod P$$

   Если остаток $m \neq 0$, остаток равномерно распределяется между процессорами.

4. Каждый процесс считывает и рассчитывает выделенную ему часть ряда.

5. Процессы с номерами $1 \ldots P-1$ пересылают свои частичные суммы нулевому процессу с помощью блокирующей функции `MPI_Send`.

6. Нулевой процесс принимает данные с помощью `MPI_Recv`, суммирует их со своей частичной суммой, фиксирует время окончания параллельного участка и выводит итоговую сумму и затраченное время T_p.

4 Результаты экспериментов

Эксперименты проводились на суперкомпьютерном кластере при числе процессов $P \in \{1, 2, 4, 8, 16\}$. Число членов ряда $N = [\text{значение}]$ подбиралось таким образом, чтобы время выполнения на одном процессоре ($P = 1$) составляло порядка 30 секунд.

P	T_p
1	30
2	—
4	—
8	—
16	—
32	—
64	—
128	—

Таблица 1: Время выполнения параллельного участка кода T_p в секундах

5 Анализ ускорения и эффективности

Ускорение S_p^* и эффективность E_p^* рассчитываются по формулам:

$$S_p^* = \frac{T_1^*}{T_p}, \qquad E_p^* = \frac{S_p^*}{P}$$

где T_1^* — время работы программы на одном процессоре, T_p — время работы программы на P процессорах.

P	Ускорение S_p^*	Эффективность E_p^*
1	1.00	1.00
2	—	—
4	—	—
8	—	—
16	—	—
32	—	—
64	—	—
128	—	—

Таблица 2: Расчетные значения ускорения S_p^* и эффективности E_p^*

5.1 Графики зависимостей ускорения и эффективности

На рисунке 1 приведена зависимость ускорения и эффективности от числа процессоров P.

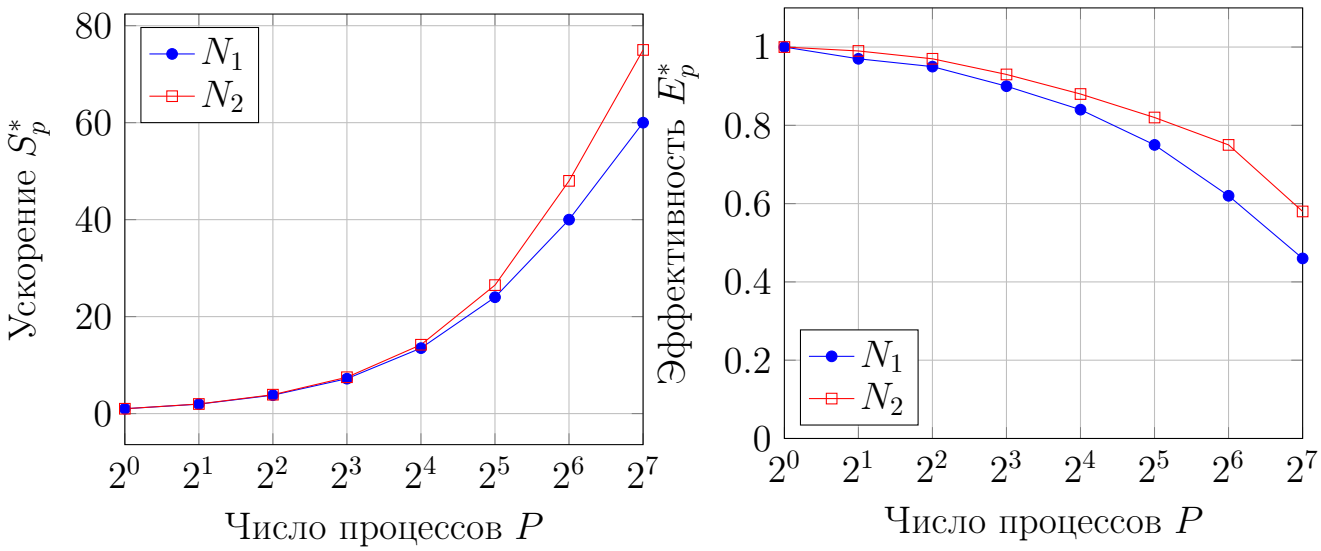


Рис. 1: Зависимость ускорения S_p^* и эффективности E_p^* от числа процессов P

Выводы

В ходе выполнения лабораторной работы:

1. Изучены базовые функции библиотеки MPI (`MPI_Init`, `MPI_Send`, `MPI_Recv`, `MPI_Wtime`) для взаимодействия процессов в системах с распределенной памятью.

2. Разработана параллельная программа на C для вычисления суммы числового ряда.

3. Проведены эксперименты при заданном числе процессов $P \in \{1, 2, \ldots\}$.

4. Проанализированы показатели ускорения и эффективности, видно, что ...

   установлено, что увеличение размера задачи приводит к росту эффективности параллельной обработки при большом числе процессоров.

Листинг программы